

Deep learning practical 2

Foundational mathematics

Practical overview

Welcome to the second deep learning practical. This practical has the following purpose:

- Introduce foundational deep learning topics.
- Review and recap the main mathematical notation and concepts used in the module.

You may already be very familiar with differentiation and can skim through this material, while others may wish to spend time understanding it and practising in PyTorch or JAX. If you finish early, please advance to the next practical.

Mathematical notation

It is recommended to follow an established mathematical style when writing deep learning papers or technical reports. The tables below are based on the optional notation in the ICLR style guide, adopted from the *Deep Learning* textbook. This distinguishes scalars, vectors, matrices and tensors by their typeface, and uses upright letters for random variables. Other conventions are also used, so define your symbols and their dimensions clearly and use them consistently.

Numbers and Arrays

a	A scalar (integer or real)
$\mathbf{a}$	A vector
$\mathbf{A}$	A matrix
$\mathbf{A}$	A tensor
$\mathbf{I}_n$	Identity matrix with n rows and n columns
$\mathbf{I}$	Identity matrix with dimensionality implied by context
$\mathbf{e}^{(i)}$	Standard basis vector $[0, \dots, 0, 1, 0, \dots, 0]^\top$ with a 1 at position i
$\text{diag}(\mathbf{a})$	A square, diagonal matrix with diagonal entries given by $\mathbf{a}$
a	A scalar random variable
$\mathbf{a}$	A vector-valued random variable
$\mathbf{A}$	A matrix-valued random variable

Sets and Graphs

$\mathbb{A}$	A set
$\mathbb{R}$	The set of real numbers
$\{0, 1\}$	The set containing 0 and 1
$\{0, 1, \dots, n\}$	The set of all integers between 0 and n
$[a, b]$	The real interval including a and b
$(a, b]$	The real interval excluding a but including b
$\mathbb{A} \setminus \mathbb{B}$	Set subtraction, i.e., the set containing the elements of $\mathbb{A}$ that are not in $\mathbb{B}$
$\mathcal{G}$	A graph
$Pa_{\mathcal{G}}(x_i)$	The parents of x_i in $\mathcal{G}$

Indexing

Mathematical indices below start at 1, whereas Python, PyTorch and JAX use indices starting at 0. The notation $\mathbf{a}_{-i}$ means omitting an element, whereas a negative Python index counts backwards from the end.

a_i	Element i of vector $\mathbf{a}$, with indexing starting at 1
$\mathbf{a}_{-i}$	Vector $\mathbf{a}$ with element i omitted
$A_{i,j}$	Element i, j of matrix $\mathbf{A}$
$\mathbf{A}_{i,:}$	Row i of matrix $\mathbf{A}$
$\mathbf{A}_{:,i}$	Column i of matrix $\mathbf{A}$
$A_{i,j,k}$	Element (i, j, k) of a 3-D tensor $\mathbf{A}$
$\mathbf{A}_{:,:,i}$	2-D slice of a 3-D tensor
$\mathbf{a}_i$	Element i of the random vector $\mathbf{a}$

Calculus

Vectors and gradients are columns throughout this practical. For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the Jacobian has one row per output and one column per input, so its shape is $m \times n$.

$\frac{dy}{dx}$	Derivative of y with respect to x
$\frac{\partial y}{\partial x}$	Partial derivative of y with respect to x
$\nabla_{\mathbf{x}} y$	Gradient of scalar y with respect to $\mathbf{x}$, with the same shape as $\mathbf{x}$
$\nabla_{\mathbf{X}} y$	Matrix of partial derivatives of scalar y , with the same shape as $\mathbf{X}$
$\nabla_{\mathbf{x}} y$	Tensor of partial derivatives of scalar y , with the same shape as $\mathbf{X}$
$\frac{\partial f}{\partial \mathbf{x}}$	Jacobian matrix $\mathbf{J} \in \mathbb{R}^{m \times n}$ of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
$\nabla_{\mathbf{x}}^2 f(\mathbf{x})$ or $\mathbf{H}_f(\mathbf{x})$	Hessian of a twice differentiable scalar f , with entry (i, j) equal to $\partial^2 f / \partial x_i \partial x_j$
$\int_{\mathbb{R}^n} f(\mathbf{x}) d\mathbf{x}$	Definite integral over all of $\mathbb{R}^n$
$\int_{\mathbb{S}} f(\mathbf{x}) d\mathbf{x}$	Definite integral with respect to $\mathbf{x}$ over the set $\mathbb{S}$

Probability and Information Theory

$P(a)$	Probability mass at value a for a discrete random variable
$p(a)$	Probability density at value a for a continuous random variable
$a \sim P$	Random variable a has distribution P
$\mathbb{E}_{x \sim P}[f(x)]$	Expectation of $f(x)$ when x has distribution P
$\text{Var}(f(x))$	Variance of $f(x)$ under the stated distribution of x
$\text{Cov}(f(x), g(x))$	Covariance of $f(x)$ and $g(x)$ under the stated distribution of x
$H(x)$	Shannon entropy of a discrete random variable x
$D_{\text{KL}}(P \ Q)$	Kullback-Leibler divergence of P and Q
$\mathcal{N}(x; \mu, \Sigma)$	Gaussian density at x , with mean μ and covariance Σ

Functions

$f : \mathbb{A} \rightarrow \mathbb{B}$	Function with domain $\mathbb{A}$ and codomain $\mathbb{B}$. Its range $f(\mathbb{A})$ may be a proper subset of $\mathbb{B}$
$f \circ g$	Composition, with $(f \circ g)(x) = f(g(x))$
$f(x; \theta)$	A function of x parametrized by θ . (Sometimes we write $f(x)$ and omit the argument θ to lighten notation)
$\log x$	Natural logarithm of x
$\sigma(x)$	Logistic sigmoid, $\frac{1}{1 + \exp(-x)}$
$\zeta(x)$	Softplus, $\log(1 + \exp(x))$
$\ x\ _p$	L^p norm of x , for $1 \leq p \leq \infty$
$\ x\ $	L^2 norm of x
x^+	Positive part of x , i.e., $\max(0, x)$
$\mathbf{1}_{\text{condition}}$	is 1 if the condition is true, 0 otherwise

Deep neural networks

Training a deep neural network usually involves estimating the gradient of a loss from small batches of examples. In stochastic minibatch gradient descent, we sample a batch from the training data and move the parameters in the negative direction of that batch's loss gradient. With learning rate $\alpha > 0$, the update for parameter θ_j is

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial \mathcal{L}_{\text{batch}}(\theta)}{\partial \theta_j}.$$

Other optimisers use the same gradients to construct different updates. Adam, for example, keeps moving averages of gradients and squared gradients to adapt the update for each parameter. Neither method guarantees that the loss will decrease at every step, especially when successive steps use different batches.

The neural network itself is a parameterised function $f_{\theta} : \mathbb{R}^m \rightarrow \mathbb{R}^n$. Its operations are usually differentiable almost everywhere, with a chosen derivative convention at points such as the corner of ReLU. Parameter counts vary widely with the task and architecture, from small teaching examples to models with billions of weights:

- 10,000–100,000 parameters in a small teaching model
- Millions or hundreds of millions of parameters in larger networks

- Billions of parameters in some large models
- 100 billion+ parameters (for example, 175 billion parameters require about 700 GB for FP32 weights or 350 GB for FP16 weights alone; gradients, optimiser state and activations require additional memory)

Differentiating a large network by hand would be time-consuming and error-prone, so libraries such as PyTorch and JAX provide automatic differentiation. They apply derivative rules to the operations in a program, usually using reverse-mode differentiation when training a network, and can run these calculations on hardware accelerators.

However, to effectively read the mathematical notation in deep learning papers and to understand what's happening inside these libraries, it is helpful to have a recap of the foundational calculus that is used.

Recap of foundational calculus

Note: *There's a lot of maths covered in the next few sections and it is completely natural for some students to feel very rusty or overwhelmed by it. For those of you, don't panic! The mathematics covered is certainly useful but mastery of it is not required to design, train and test deep neural networks with modern tooling.*

Derivatives

This section starts by recapping the definition of derivative for a simple real-valued function and then we extend this definition to vector-valued functions that are typically used in neural networks.

Differentiation is the operation of finding the derivative of a function.

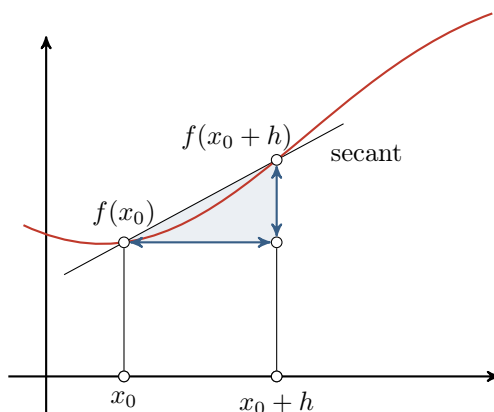
At an interior point x , the derivative of a function $f : \mathbb{R} \rightarrow \mathbb{R}$ is defined by the following limit, provided it exists and has the same finite value as h approaches zero from either side:

$$f'(x) = \frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

- The derivative measures how a function's output changes in response to a small change in its input.
 - In deep learning, derivatives of the loss tell us how to adjust a neural network's parameters to reduce prediction error locally.
- It is the input increment h that tends to zero. The ratio approaches the derivative, which need not be zero.
 - Basically, as we take h to zero, we end up with a tangent line whose slope is the derivative.

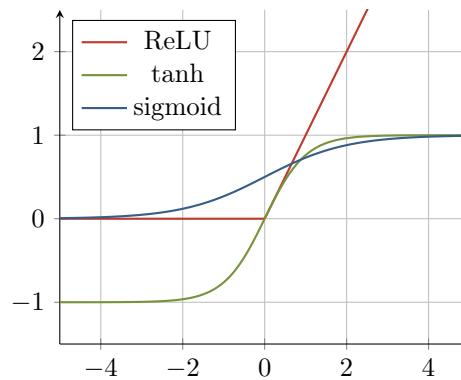
Exercise: visit: <https://www.desmos.com/calculator>

- Type $f(x) = \tanh(x)$, type $f'(x)$ for the first derivative, $f''(x)$ for the second...



Common derivatives in deep learning

In deep learning, we often chain together linear transformations with nonlinear ‘activation’ functions. The following are common choices for non-linearities with their associated derivatives:



$$1. \text{ReLU}'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x < 0 \end{cases}$$

At $x = 0$, the left and right derivatives differ, so ReLU is not differentiable there. PyTorch's ReLU uses 0 in backpropagation at this point, which is a valid subgradient.

$$2. \tanh'(x) = 1 - \tanh^2(x)$$

$$3. \text{sigmoid}'(x) = \text{sigmoid}(x) \cdot (1 - \text{sigmoid}(x))$$

$$4. \sin'(x) = \cos(x)$$

Rules of differentiation

The following rules apply where the derivatives exist. The quotient rule also requires $g(x) \neq 0$, and the power rule applies on the domain where x^n is differentiable.

The **sum rule** is defined as:

$$(f(x) + g(x))' = f'(x) + g'(x)$$

The **product rule** is defined as:

$$(f(x)g(x))' = f'(x)g(x) + f(x)g'(x)$$

The **quotient rule** is defined as:

$$\left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)g(x) - f(x)g'(x)}{(g(x))^2}$$

The **chain rule** is defined as:

$$(g(f(x)))' = (g \circ f)'(x) = g'(f(x))f'(x)$$

The **power rule** is defined as:

$$(x^n)' = nx^{n-1}$$

Exercise:

What is the derivative of $h(x) = \sin(x^2)$?

$$g(x) = \sin(x)$$

$$g'(x) = \cos(x)$$

$$f(x) = x^2$$

$$f'(x) = 2x$$

▷ power rule

$$h'(x) = g'(f(x))f'(x)$$

▷ chain rule

$$= \cos(x^2)2x$$

Explanation:

- We want to differentiate the function $h(x) = \sin(x^2)$

- First, we can break this down into two functions:
 1. $g(x) = \sin(x)$
 2. $f(x) = x^2$
- We can calculate their derivatives respectively.
 - The derivative of $g(x) = \sin(x)$ is $g'(x) = \cos(x)$
 - The derivative of $f(x) = x^2$ is, by use of the power rule, $f'(x) = 2x^1 = 2x$
- We can then substitute these into the definition of the chain rule.
- Which gives the answer $\cos(x^2)2x$ —you may want to check these steps in desmos.

Partial derivatives

We can extend these rules from functions of one variable to functions of several variables. A partial derivative varies one input while holding the others fixed.

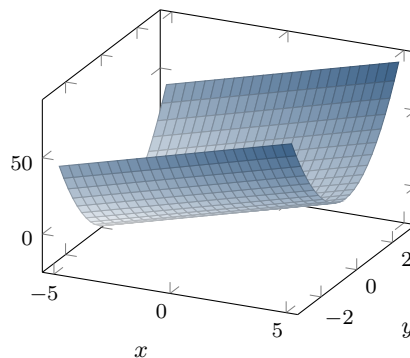
For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ of n variables $x_1, \dots, x_n$, the partial derivatives are defined as

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= \lim_{h \rightarrow 0} \frac{f(x_1 + h, x_2, \dots, x_n) - f(\mathbf{x})}{h} \\ &\vdots \\ \frac{\partial f}{\partial x_n} &= \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{n-1}, x_n + h) - f(\mathbf{x})}{h} \end{aligned}$$

whenever the limits exist. We collect these derivatives into a column vector called the gradient of f with respect to $\mathbf{x}$:

$$\nabla_{\mathbf{x}} f = \left[\frac{\partial f(\mathbf{x})}{\partial x_1} \quad \dots \quad \frac{\partial f(\mathbf{x})}{\partial x_n} \right]^{\top} \in \mathbb{R}^{n \times 1}.$$

Example function: $f(x, y) = 4x + 7y^2$



- For example, $f(x, y) = 4x + 7y^2$ has two inputs and one scalar output, so it maps from $\mathbb{R}^2$ to $\mathbb{R}$.
- Therefore the function admits two partial derivatives:
 1. $\frac{\partial f}{\partial x}$ read: partial derivative of f with respect to x
 2. $\frac{\partial f}{\partial y}$ read: partial derivative of f with respect to y

Calculating the partial derivatives:

- For a given function $f(x, y) = 4x + 7y^2$ go through each input (in our case x and y)
 - Write down $\frac{\partial f}{\partial x} =$
 - Write down $\frac{\partial f}{\partial y} =$
- Look at the variable in the denominator
- See if it exists in any of the terms, if it doesn't that's '0'

- Otherwise, differentiate with respect to that variable while treating the other variables as constants.

This gives

- $\frac{\partial f}{\partial x} = 4 + 0$
- $\frac{\partial f}{\partial y} = 0 + 14y$

Exercise:

Calculate $\nabla_{\mathbf{x}} f$ for $f(x, y, z) = \cos(x) + z^3 - x^2y$:

- Write down $\frac{\partial f}{\partial x} = \dots \frac{\partial f}{\partial y} = \dots \frac{\partial f}{\partial z} = \dots$
- Go through each variable in the denominator
- If it exists, differentiate it, if it doesn't that's a 0
- This gives:

$$\begin{aligned} - \frac{\partial f}{\partial x} &= -\sin(x) + 0 - 2xy \\ - \frac{\partial f}{\partial y} &= 0 + 0 - x^2 \\ - \frac{\partial f}{\partial z} &= 0 + 3z^2 - 0 \end{aligned}$$

Combining the three partial derivatives gives

$$\nabla_{\mathbf{x}} f = \begin{bmatrix} -\sin x - 2xy \\ -x^2 \\ 3z^2 \end{bmatrix}, \quad \mathbf{x} = (x, y, z)^{\top}.$$

The same convention applies to a scalar loss with many inputs: its gradient has one entry per input and is written as a column vector.

Further practice and checking:

Visit: <https://www.wolframalpha.com> and type in $f(x,y,z) = \dots$ solve and check answers, especially for any functions that you're not sure about.

Rules of partial differentiation

For differentiable scalar functions $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$, the sum and product rules can be written using gradients. For the chain rule below, the outer function $g : \mathbb{R} \rightarrow \mathbb{R}$ takes the scalar output of f .

The **sum rule** is defined as:

$$\nabla_{\mathbf{x}}(f(\mathbf{x}) + g(\mathbf{x})) = \nabla_{\mathbf{x}}f(\mathbf{x}) + \nabla_{\mathbf{x}}g(\mathbf{x})$$

The **product rule** is defined as:

$$\nabla_{\mathbf{x}}(f(\mathbf{x})g(\mathbf{x})) = g(\mathbf{x})\nabla_{\mathbf{x}}f(\mathbf{x}) + f(\mathbf{x})\nabla_{\mathbf{x}}g(\mathbf{x})$$

The **chain rule** is defined as:

$$\nabla_{\mathbf{x}}(g \circ f)(\mathbf{x}) = g'(f(\mathbf{x}))\nabla_{\mathbf{x}}f(\mathbf{x})$$

Exercise:

Calculate the partial derivative of $z^4 - \sin(y^2 + x)$ w.r.t. y

By use of the chain rule

$$\frac{\partial}{\partial y}(z^4 - \sin(y^2 + x)) = -2y \cos(y^2 + x)$$

Also we can calculate for x and z

$$\begin{aligned} \frac{\partial}{\partial x}(z^4 - \sin(y^2 + x)) &= -\cos(y^2 + x) \\ \frac{\partial}{\partial z}(z^4 - \sin(y^2 + x)) &= 4z^3 \end{aligned}$$

Try your own and test your answers on <https://www.wolframalpha.com> e.g. type $d/dy - \sin(y^2 + x)$

- For each input coordinate, the usual differentiation rules apply with the remaining coordinates held fixed.
- Bold $\mathbf{x}$ denotes the vector of inputs, while x_i denotes one scalar entry. The gradient collects all the coordinate derivatives.
- When intermediate functions have vector outputs, the chain rule uses Jacobian matrices, introduced next. Their multiplication order matters.
- The notation $\frac{\partial}{\partial y}(\dots)$ means differentiating the expression in brackets with respect to the scalar y .
- Another shorter example:
 - You may see $\frac{\partial}{\partial y} 3yx^2$
 - which means the partial derivative of $3yx^2$ with respect to y
 - $\frac{\partial}{\partial y} 3yx^2 = 3x^2$

Jacobian matrix

- In the previous example, we looked at differentiating functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$
- Now, we want to extend our definition to differentiating vector-valued functions of domain $\mathbb{R}^n$ to the co-domain $\mathbb{R}^m$
- This is an easy extension, basically you calculate the partial derivatives for each of the functions individually with respect to each of the inputs
 - which gives an $m \times n$ matrix of partial derivatives

Jacobian matrix definition: The collection of all first-order partial derivatives of a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$

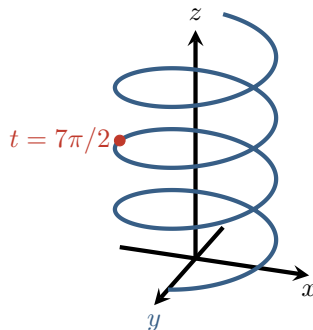
$$\mathbf{J}_f(\mathbf{x}) = \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_m(\mathbf{x})}{\partial x_1} & \dots & \frac{\partial f_m(\mathbf{x})}{\partial x_n} \end{bmatrix},$$

$$\mathbf{J}_f(i, j) = \frac{\partial f_i}{\partial x_j}$$

For a scalar-valued function, the Jacobian is a row and the gradient is its transpose: $\mathbf{J}_f = (\nabla_{\mathbf{x}} f)^\top$. For differentiable $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $g : \mathbb{R}^m \rightarrow \mathbb{R}^p$, the chain rule becomes

$$\mathbf{J}_{g \circ f}(\mathbf{x}) = \mathbf{J}_g(f(\mathbf{x})) \mathbf{J}_f(\mathbf{x}).$$

Example function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$
 $f(t, s) = \langle \sin(t) + s, \cos(t), \frac{6t}{\pi} \rangle$



Example:

- For each fixed s , this function parameterises a 3D helix. The plot shows $s = 0$.
- This function has two input variables, t and s , and 3 outputs corresponding to the Cartesian x, y, z coordinates.

- So it maps from $\mathbb{R}^2 \rightarrow \mathbb{R}^3$
- as you increase t , the x, y, z coordinates move in a 3D helical curve
- as you change s , you can see this just translates the helix along the x -axis

Calculate the Jacobian for $f(t, s) = \langle \sin(t) + s, \cos(t), \frac{6t}{\pi} \rangle$ w.r.t. (t, s)

- The Jacobian is an $\mathbb{R}^{m \times n}$ matrix. In this case $\mathbb{R}^{3 \times 2}$ where

$$\mathbf{J}_f = \begin{bmatrix} \frac{\partial \sin(t)+s}{\partial t} & \frac{\partial \sin(t)+s}{\partial s} \\ \frac{\partial \cos(t)}{\partial t} & \frac{\partial \cos(t)}{\partial s} \\ \frac{\partial}{\partial t} \left(\frac{6t}{\pi} \right) & \frac{\partial}{\partial s} \left(\frac{6t}{\pi} \right) \end{bmatrix} = \begin{bmatrix} \cos(t) & 1 \\ -\sin(t) & 0 \\ 6/\pi & 0 \end{bmatrix}$$

- As an exercise, cover the right side of the above and complete it yourself

Mathematics for neural networks

We can differentiate a neural network by applying the chain rule to its sequence of layers wherever the required derivatives exist.

- These are just compositions of many layers of functions, which need to be broken down to their foundations, so we can then just use the chain rule one at a time.
- By doing this, we will be able to compute the gradient of the network objective function with respect to each of its parameters.
 - We can then update each parameter in the negative direction of the gradient, such as to minimise the error of whole network.

Affine maps & vector summation

A basic neural-network layer applies a learned weight matrix $\mathbf{W}$ and bias vector $\mathbf{b}$. It is often called a linear layer, although the bias makes the map affine.

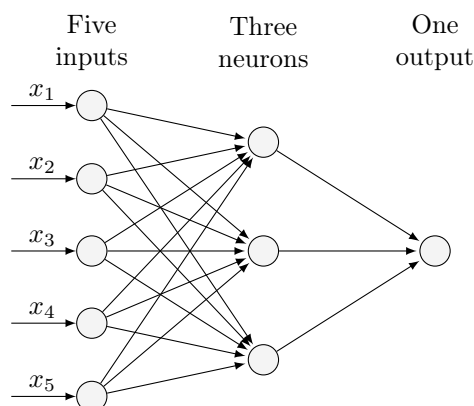
More formally, an affine map is a function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$

$$f(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\frac{\partial}{\partial \mathbf{x}} (\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{W}$$

Example neural network

- The diagram underneath represents a very simple example feed-forward neural network.
 - It has 5 inputs.
 - Then a hidden layer with 3 neurons.
 - * Each neuron sums the inputs multiplied by its own weights, then adds its own scalar bias.
 - * Collecting the three outputs gives an affine map, before applying any nonlinear activation.



- So there are $5 \times 3 = 15$ edges which we represent as a 3×5 weight matrix $\mathbf{W}$
- the operation is to then multiply this by the $\mathbb{R}^5$ input vector $\mathbf{x}$ which gives an $\mathbb{R}^3$ output vector, we then add the $\mathbb{R}^3$ bias vector to this
- if you differentiate that w.r.t $\mathbf{x}$, you will see it's just $\mathbf{W}$

A simple loss function:

- To study a loss on these three values, temporarily treat the hidden-layer outputs as predictions, omitting the final one-output layer in the diagram.
- Ultimately, we need some cost function which somehow measures the quality of the outputs' prediction and spits out a single value, which is the network error
 - for example we could take the Euclidean distance between those three outputs and some target vector, which gives a single scalar
- The cost functions usually at one point requires a vector summation, and we can see the Jacobian of the summation function is just a row vector of 1's.

More formally, vector summation is defined $f: \mathbb{R}^n \rightarrow \mathbb{R}$

$$f(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$\mathbf{J}_f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right] = [1, 1, \dots, 1]$$

For the squared L_2 norm, let $r(\mathbf{x}) = \sum_{k=1}^n x_k^2$. Differentiating one coordinate at a time gives

$$\frac{\partial r(\mathbf{x})}{\partial x_j} = \frac{\partial}{\partial x_j} \sum_{k=1}^n x_k^2 = \sum_{k=1}^n \underbrace{\frac{\partial}{\partial x_j} x_k^2}_{\substack{=0, & \text{if } j \neq k, \\ =2x_j, & \text{if } j=k}} = 2x_j$$

The full gradient is $\nabla_{\mathbf{x}} r = [2x_1, \dots, 2x_n]^\top = 2\mathbf{x}$.

Computational graphs

Deep neural networks get very large and complicated very quickly, where we need an automatic way to differentiate them efficiently.

- The most common algorithm we use is called backpropagation, which essentially implements the reverse mode of differentiation.

Consider a neural network with one linear layer:

$$f(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b},$$

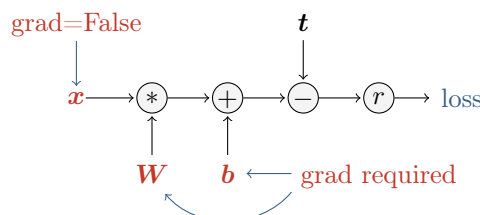
with $\mathbf{W} \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$. Define r on its m -dimensional prediction error as the squared L_2 (Euclidean) norm:

$$r(\mathbf{e}) = \|\mathbf{e}\|_2^2 = \sum_{i=1}^m e_i^2,$$

so, for a fixed input $\mathbf{x}$ and target vector $\mathbf{t} \in \mathbb{R}^m$, the scalar loss is

$$\mathcal{L}(\mathbf{W}, \mathbf{b}) = r(f(\mathbf{x}) - \mathbf{t}) = \|(\mathbf{W}\mathbf{x} + \mathbf{b}) - \mathbf{t}\|_2^2.$$

This is implemented as a computational graph

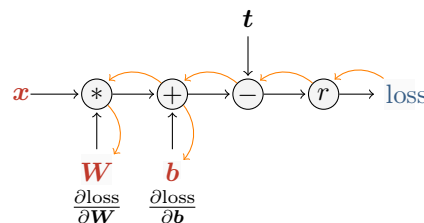


- We start by representing our function as a computational graph
 - this is a directed graph
 - nodes are operations
 - variables are fed into the nodes
 - the output is the loss, which is a single value that we wish to minimise
- The example graph you see is just an extension to what we’ve already differentiated manually.
 - Instead of summing the outputs, we have an actual loss function.
 - This measure outputs squared Euclidean distance between the predictions and the target labels t
 - What we want to do is compute the gradients of the loss w.r.t the network parameters, which are leaf variables W and b .

Backpropagation: reverse accumulation

Backpropagation is reverse-mode automatic differentiation, which is especially efficient when a function has many inputs and few outputs. For training, the inputs being differentiated are the model parameters and the output is a scalar loss, even if the network itself predicts many values. In PyTorch, we:

1. set `requires_grad=True` for any parameters we want to optimise (W and b)
2. calculate the forward pass (feed the network x and see the prediction) then calculate the loss (error function)
 - PyTorch records differentiable operations and saves the intermediate values needed for their backward calculations, provided gradient tracking is enabled.
3. call `loss.backward()` to traverse the graph in reverse and accumulate the loss derivatives in the parameters’ `.grad` fields. This calculates gradients without changing the parameter values.



Programming exercise: learning two spirals

One toy example is getting a neural network to learn which spiral a point belongs to from just its two coordinates. To do this, we give the model labelled examples, then watch its predictions change as gradient descent adjusts the weights. A straight line cannot separate these spirals, so the example also lets us investigate the role of nonlinear activation functions.

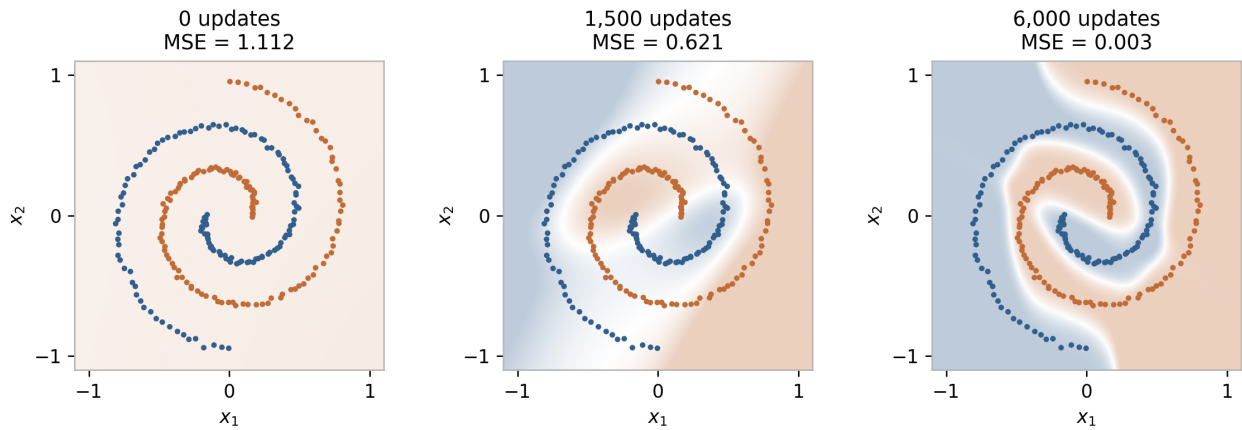


Figure 1: Learning from the same 256 points. Dots show the labels: orange is +1 and blue is -1. Background colour shows the network's output, fading to white near zero and reaching the end colours at ± 1 . As training reduces the mean squared error, the background colour comes to match the dots along both spirals.

Work through `dl-practical2.py` in IPython or VS Code. Its plotting functions produce the figures and a replay of training with the learning code given below. This example runs on the CPU and generates the data directly.

Data generation. We construct one spiral from an increasing angle and radius, taking the other as its reflection through the origin. Adding a little noise gives us varied observations of each arm, and following best practice, a separate set of held out points lets us check the performance of the trained network.

```
import torch
from torch import nn
# Run the plotting-functions cell first in dl-practical2.py.

torch.set_num_threads(2)
torch.manual_seed(42)

def spirals(n):
    t = torch.linspace(0, 1, n)
    angle = 2.5 * torch.pi * t
    radius = 0.15 + 0.8 * t
    arm = torch.stack((radius * angle.cos(), radius * angle.sin()), dim=1)
    X = torch.cat((arm, -arm)) + 0.01 * torch.randn(2 * n, 2)
    y = torch.cat((torch.ones(n, 1), -torch.ones(n, 1)))
    return X, y

X, y = spirals(128)
X_test, y_test = spirals(512)
```

Exercise: Sketch the shapes of `arm`, `X` and `y` as rows and columns. Which axis does `stack` create, and which existing axis does `cat` extend? What shape would `model(X) - y` have if `y` were `(256,)` instead of `(256, 1)`?

Building the network. Each hidden layer combines its inputs with a learned weight matrix and bias, then applies Tanh to each entry. Composing these operations lets the network represent a curved separation between the classes. Its last hidden layer produces two learned features $\mathbf{h} = (h_1, h_2)^\top$, which the output layer combines into a single score:

$$\mathbf{h} = f_{\theta}(\mathbf{x}), \quad s(\mathbf{x}) = w_1 h_1 + w_2 h_2 + b.$$

A positive score predicts orange and a negative score predicts blue. The training labels are +1 and -1, so we will use squared error to encourage the corresponding scores.

```

torch.manual_seed(7)
width = 32
model = nn.Sequential(
    nn.Linear(2, width), nn.Tanh(),
    nn.Linear(width, width), nn.Tanh(),
    nn.Linear(width, 2), nn.Tanh(),
    nn.Linear(2, 1),
)
print(sum(p.numel() for p in model.parameters()), "parameters")

```

Exercise: Calculate the number of parameters in each Linear layer and check their total against the program's output. Which operations have no learned parameters?

Calculating gradients. Averaging the squared errors over the $N = 256$ training points gives

$$L = \frac{1}{N} \sum_{i=1}^N (s_i - y_i)^2, \quad \frac{\partial L}{\partial w_j} = \frac{2}{N} \sum_{i=1}^N (s_i - y_i) h_{ij}.$$

The derivative tells us how a small change to an output weight affects the loss. Calling `backward()` applies the chain rule through the whole network, calculating a derivative for every parameter.

```

score = model(X)
loss = (score - y).square().mean()
loss.backward()
print(model[-1].weight.grad)
model.zero_grad()

```

Exercise: Use `model[-1](X)` to obtain the hidden features and check the printed derivatives against the expression above. Has `backward()` changed the weights? Explain the need to clear gradients before calculating them again.

Training and inspecting the predictions. Use gradient descent to update every parameter tensor according to $\theta \leftarrow \theta - 0.1 \nabla_{\theta} L$. The learning rate 0.1 controls the size of each update; repeating the forward pass and gradient calculation then gives a new direction from the current weights.

```

frames = [snapshot(model, X, y, step=0)]
for step in range(1, 6001):
    loss = (model(X) - y).square().mean()
    loss.backward()
    with torch.no_grad():
        for p in model.parameters():
            p -= 0.1 * p.grad
            p.grad.zero_()
    if step % 150 == 0:
        frames.append(snapshot(model, X, y, step))
show_progress(frames, X, y)
replay(frames, X, y)

```

Open `spirals.html` in a browser and move the slider through training. Compare the predictions with the loss curve: where do the colours still disagree, and when does the fit improve most quickly? The supplied run reaches a mean squared error of approximately 0.0034 after 6000 updates.

Checking new observations. After fitting the training points, evaluate predictions on the observations we set aside earlier.

```

with torch.no_grad():
    prediction = model(X_test)
    accuracy = ((prediction > 0) == (y_test > 0)).float().mean()
print(f"Test accuracy: {accuracy:.1%}")

```

The supplied run classifies all 1024 held-out points correctly. These observations follow the same spiral-generating process, so this result measures performance on that particular task; the background far from the spirals has no labelled observations against which to check it. Once you use the held-out set to choose a model, treat it as validation data and generate fresh observations for a final test.

Experiments:

1. Remove all three Tanh operations and rerun from the model-definition block. Predict the shape of the separation before training, then compare the predictions and loss with the original network. Explain the result by composing the affine maps.

2. Restore Tanh and try widths 8 and 16. Compare how quickly each network learns the spirals, and record its final loss, held-out accuracy and parameter count. Can two networks have the same accuracy but different losses? How small can you make the network while preserving a good fit?

Visualising gradient descent. Training changes many weights together, which makes its path difficult to draw. To examine the same update rule in two dimensions, keep the trained hidden layers and bias fixed, then fit a fresh pair of output weights starting at $\mathbf{w} = \mathbf{0}$. The hidden features already describe the spirals, so these two weights can quickly recover a good classifier. Use the original width-32 network for this comparison.

In the left panel below, each location represents a choice of (w_1, w_2) , and each grey contour joins choices with the same loss. A sufficiently small step in the negative gradient direction moves towards lower contours. The right panel shows the predictions obtained from the final pair of weights, using the original input coordinates. Calling `detach()` treats the learned features and bias as constants in the new calculation. A separate tensor `w` holds the two weights we will optimise, leaving the trained model available for comparison.

```
H = model[:-1](X).detach()
b = model[-1].bias.detach()
w = torch.zeros(2, 1, requires_grad=True)
path = [w.detach().clone()]
for step in range(20):
    loss = (H @ w + b - y).square().mean()
    loss.backward()
    with torch.no_grad():
        w -= 0.2 * w.grad
        w.grad.zero_()
    path.append(w.detach().clone())
show_weights(model, X, y, path)
```

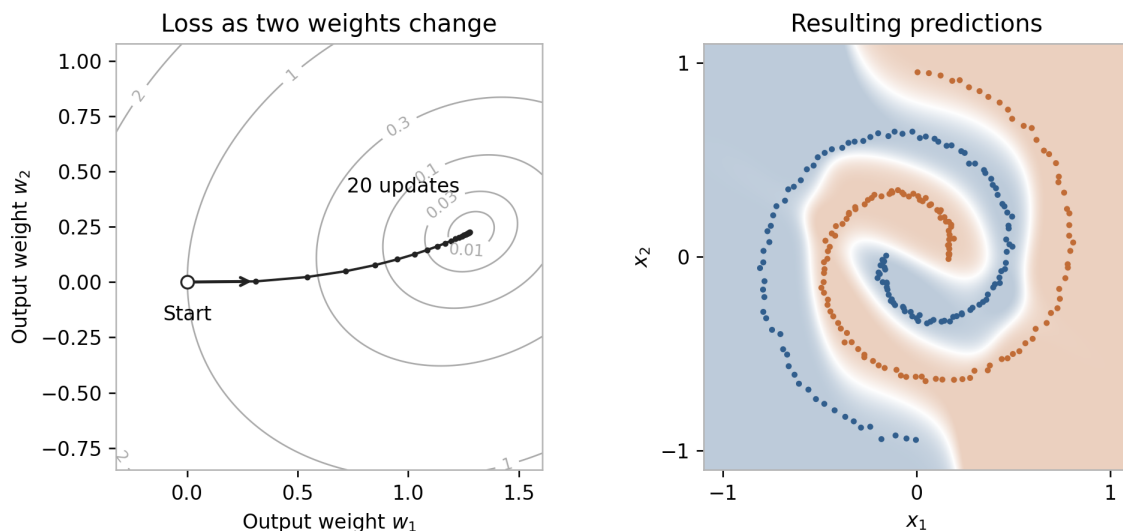


Figure 2: Gradient descent on two output weights. The black dots mark successive weight pairs, starting at $(0,0)$; contour labels give the mean squared error. After 20 updates, these weights produce the predictions on the right. The input points, hidden layers and bias stay fixed throughout this experiment.

Exercises:

1. Before running the code, predict the output when both new weights are zero. How does the contour plot explain the direction of the first update?
2. Change the learning rate from 0.2 to 0.02 and then 0.8, restarting from zero each time. Compare the paths and final losses after 20 updates. Explain why the step size affects how quickly the weights approach the minimum.
3. Why can two adjustable weights now separate the spirals, while the network with all its Tanh operations removed could not?

Forward accumulation

Forward-mode automatic differentiation propagates a directional derivative from input to output. One sweep computes a Jacobian–vector product without storing the full Jacobian.

- For $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, computing all columns of the Jacobian generally needs n forward sweeps. Forward mode is useful when n is small relative to m .
- A million-by-million Jacobian has 10^{12} entries and needs 4 TB in float32 if stored explicitly. Neither a Jacobian–vector product nor a vector–Jacobian product requires this full matrix to be stored.
- A neural-network training loss is usually scalar. Reverse mode can compute its gradient with respect to all parameters in one reverse sweep, which makes it preferable when there are many parameters.

Reverse accumulation:

Backpropagation reuses intermediate derivatives as it traverses the graph, rather than differentiating the entire expression separately for every parameter. For the usual neural-network operations, calculating the gradient of a scalar loss costs a small constant multiple of its forward evaluation, even when there are millions of parameters. This explains its usefulness without implying that it is the cheapest possible algorithm for every computation.

Once those gradients are available, an optimiser uses them to update the parameters. Backpropagation and optimisation are separate steps, as illustrated by `loss.backward()` and the subsequent weight subtraction in the spiral example.

Discussion points and open research questions:

- Why do we often need thousands of optimisation steps in deep learning? Is spending millions on training mathematically necessary?
- How would the batch size influence the number of gradient-update steps required?
- Could we learn to ‘store’ datasets in 1-10 optimization steps? How would you define and contrast the concepts of ‘compression’ and ‘generalisation’ using deep neural networks?

Practice in PyTorch and JAX: ‘Dive into deep learning’ sections 2.4 - 2.7

We recommend that you follow sections 2.4 through to 2.7 in the “Dive into deep learning” book, available online, practicing examples in PyTorch (and optionally JAX):

https://d2l.ai/chapter_preliminaries/calculus.html

- If you finish this material early, we would recommend reading ahead in this book, such as chapters 4-5, again practicing the examples in both PyTorch and JAX.